

# Artificial Intelligence

AI2002

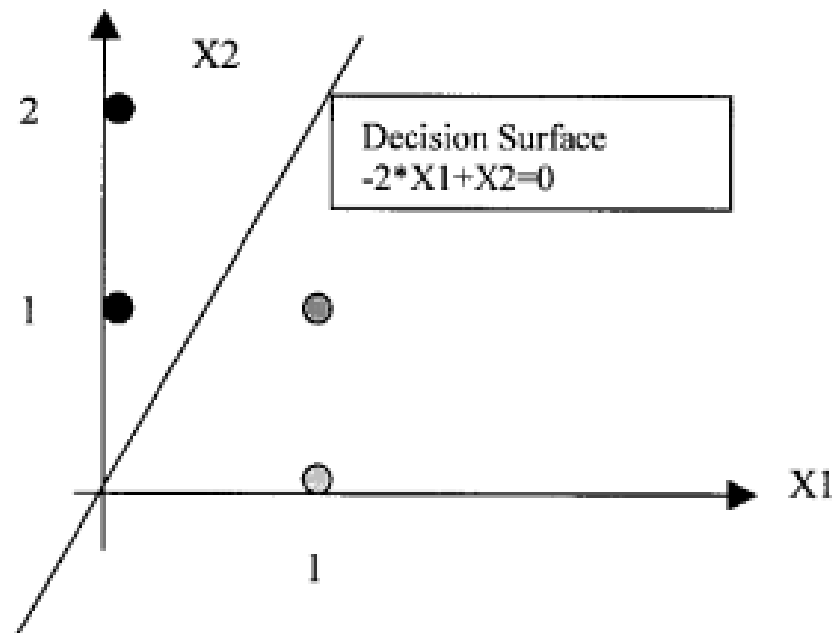
Rushda Muneer

Spring 2026

# Multilayer Perceptron

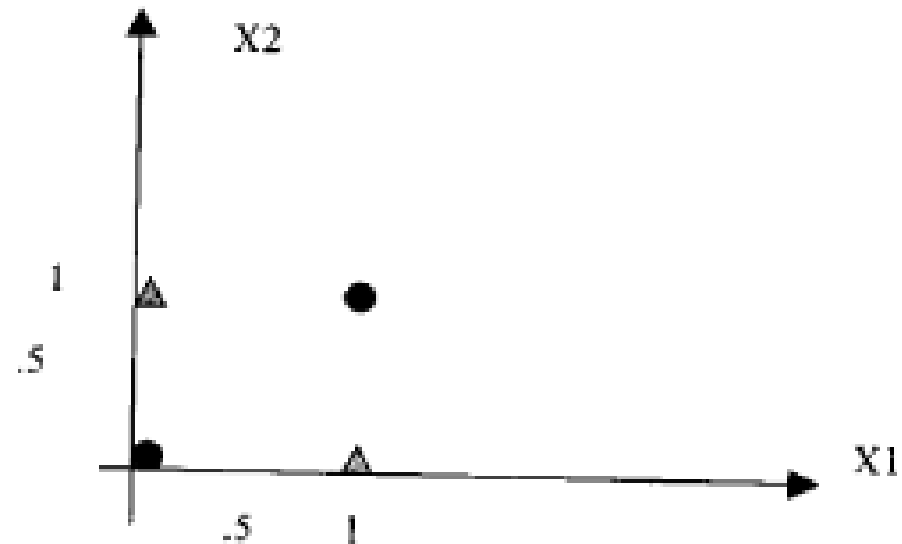
# Linearly Separable Problems

- If the data can be correctly divided into two categories by a line or hyper-plane



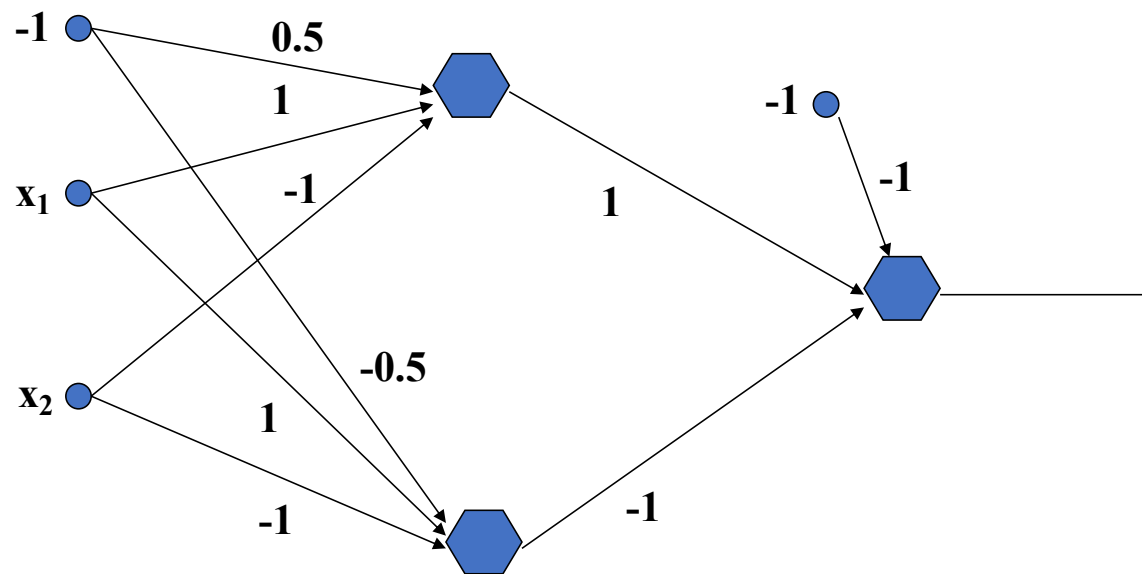
# Non Linear Problems

- If the data cannot be correctly separated by a single line or plane, then it is not linearly separable. A single layer of neurons cannot classify the input patterns that are not linearly separable; e.g. exclusive OR problem (XOR)



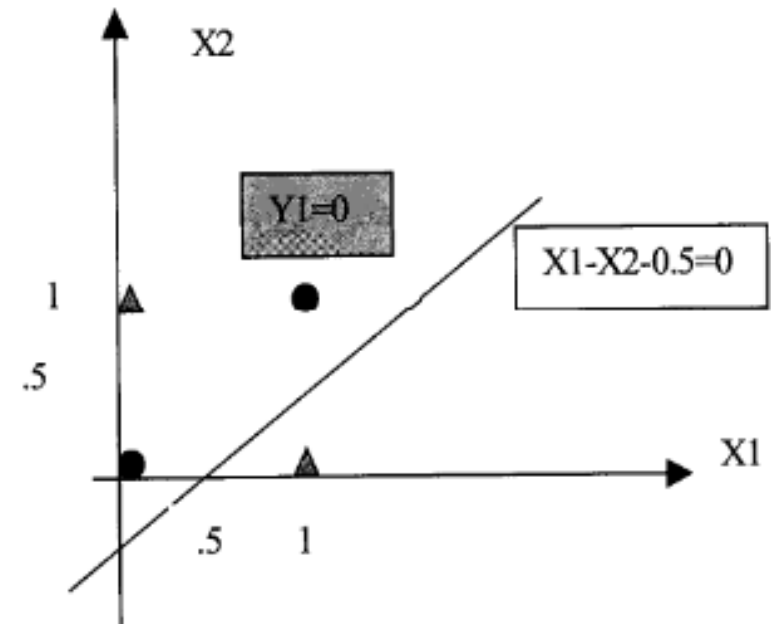
# Non Linear Problems

- For such problems we need two or more layers of neurons



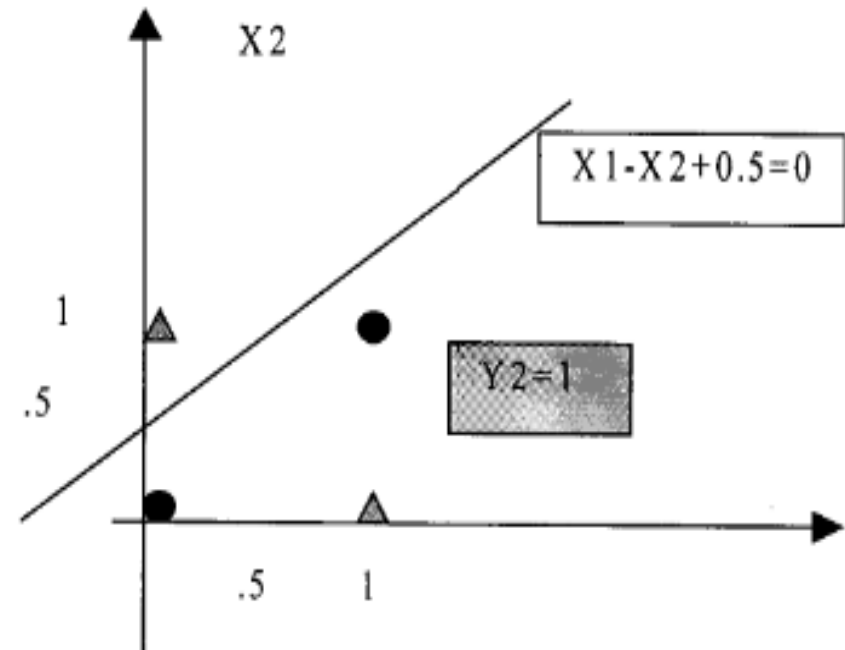
# Non Linear Problems XOR

- The first neuron's output is
  - Point (0, 0) => Sum = -0.5 and Y1 = 0
  - Point (0, 1) => Sum = -1.5 and Y1 = 0
  - Point (1, 0) => Sum = 0.5 and Y1 = 1
  - Point (1, 1) => Sum = -0.5 and Y1 = 0
- 
- It places a line



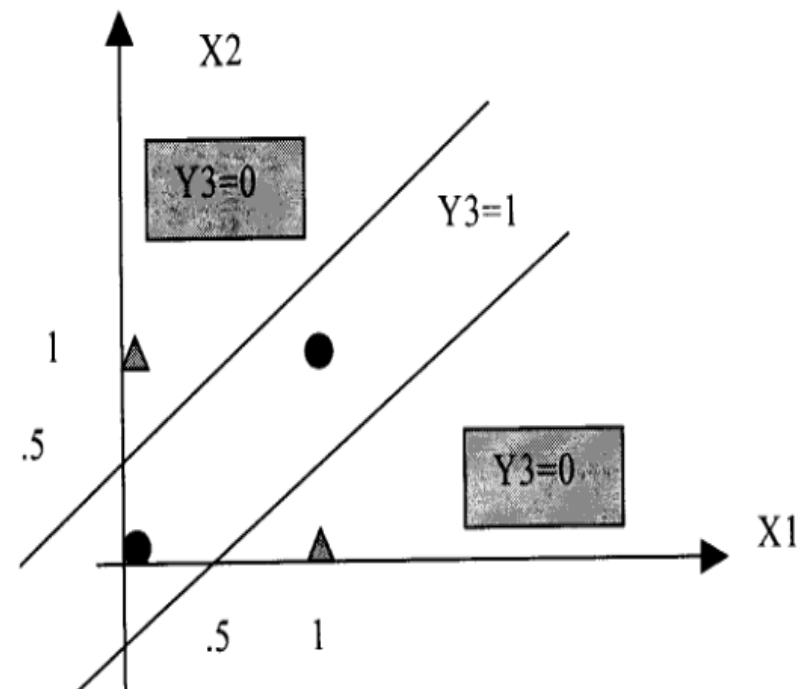
# Non Linear Problems

- The second neuron's output is
- Point (0, 0) => Sum = 0.5 and Y2 = 1
- Point (0, 1) => Sum = -0.5 and Y2 = 0
- Point (1, 0) => Sum = 1.5 and Y2 = 1
- Point (1, 1) => Sum = 0.5 and Y2 = 1
- It places a line



# Non Linear Problems

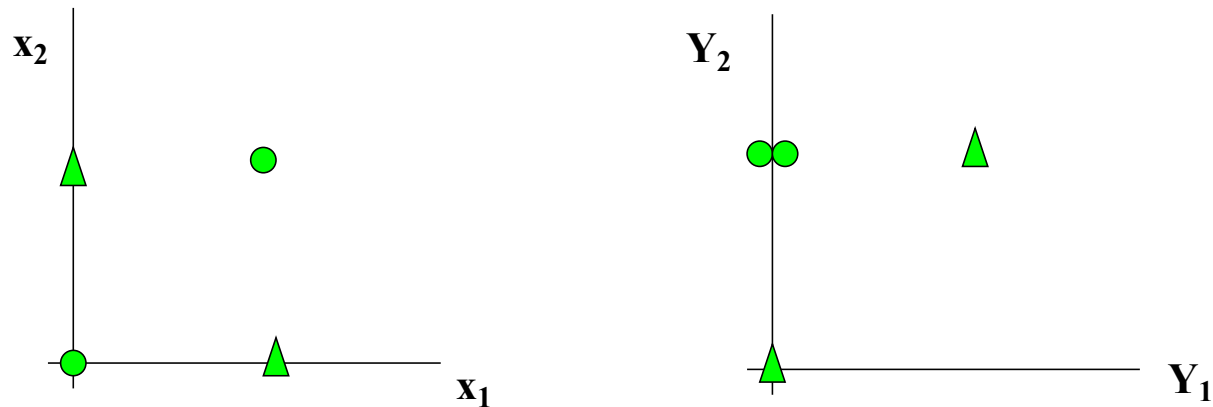
- The input sum for the neuron of the 2nd layer  
=  $Y1 - Y2 + 1$
  - For point (0,1)  $Y1 = 0$ ,  $Y2 = 0$ , Sum = 1,  $Y3 = 1$
  - For point (1,0)  $Y1 = 1$ ,  $Y2 = 1$ , Sum = 1,  $Y3 = 1$
  - For point (0,0)  $Y1 = 0$ ,  $Y2 = 1$ , Sum = 0,  $Y3 = 0$
  - For point (1,1)  $Y1 = 0$ ,  $Y2 = 1$ , Sum = 0,  $Y3 = 0$
- 
- Thus the neuron of the 2nd layer separates the data correctly
  - Let threshold be 0.5 for step function





# Non Linear Problems

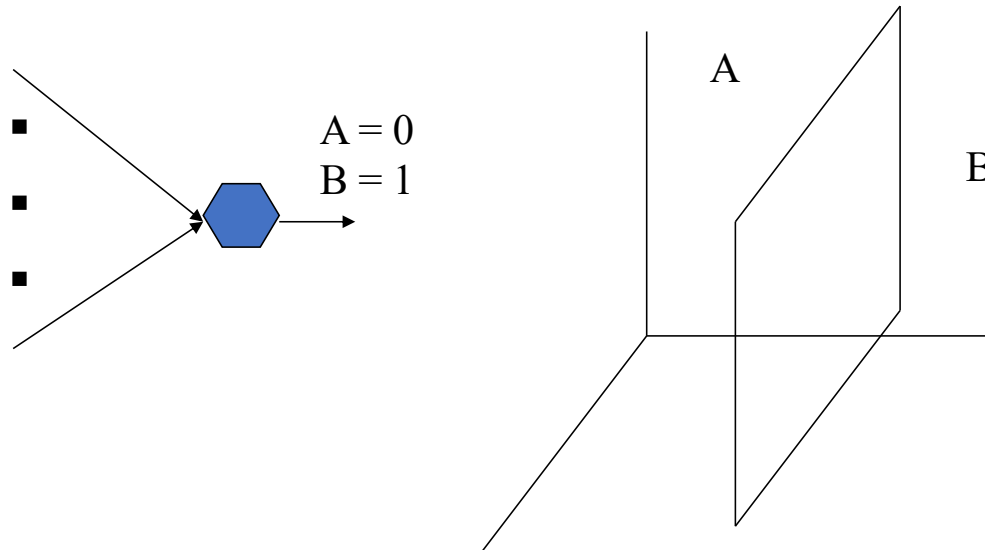
- The first layer projects the input data ( $x_1, x_2$ ) into another dimension ( $Y_1, Y_2$ )



- In dimension ( $Y_1, Y_2$ ), the data becomes linearly separable

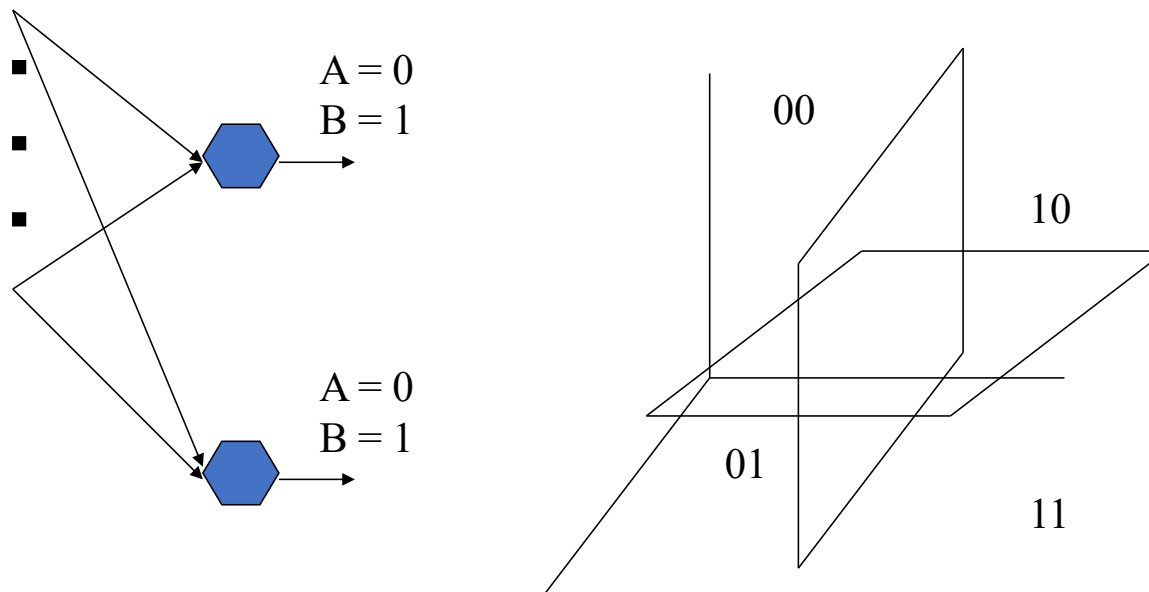
# Multi Layer Perceptron: Utilization

- Hence a single neuron with unit step activation function can classify the input into two categories



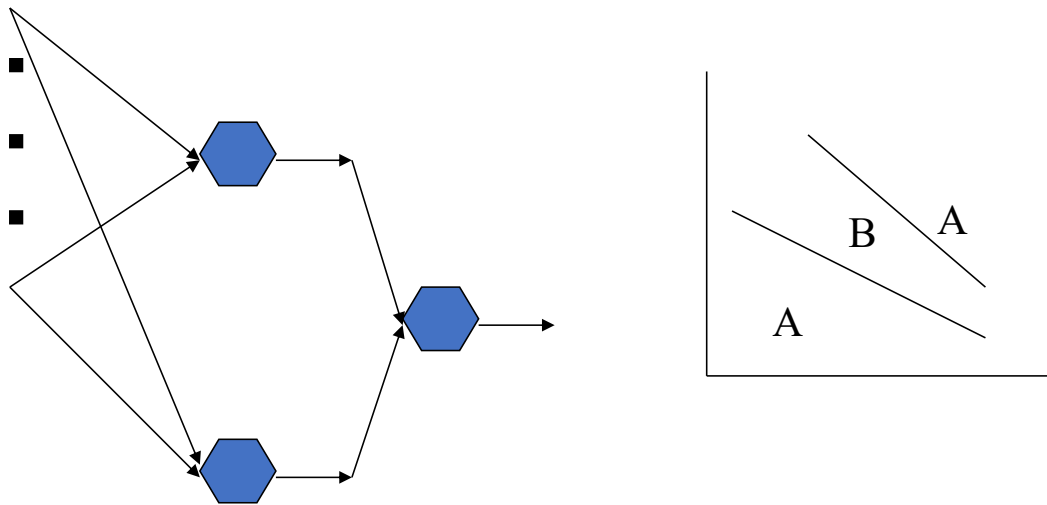
# Multi Layer Perceptron: Utilization

- Two neurons with unit step activation function can classify the input into four categories



# Multi Layer Perceptron: Utilization

- Two layers of neurons can classify complex types of inputs



# Common Activation Functions

## 1. Binary Step Function

(a “threshold” or “Heaviside” function)

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

## 2. Bipolar Step Function

## 3. Binary Sigmoid Function

(Logistic Sigmoid)

$$f(x) = \begin{cases} -1 & \text{if } x < 0 \\ 1 & \text{if } x \geq 0 \end{cases}$$

$$f(x) = \frac{1}{1 + \exp(-\sigma x)}; f'(x) = \sigma f(x)[1 - f(x)]$$

## 3. Bipolar Sigmoid

$$g(x) = \frac{1 - \exp(-\sigma x)}{1 + \exp(-\sigma x)}; g'(x) = \frac{\sigma}{2} [1 + g(x)][1 - g(x)]$$

## 4. Hyperbolic - Tangent

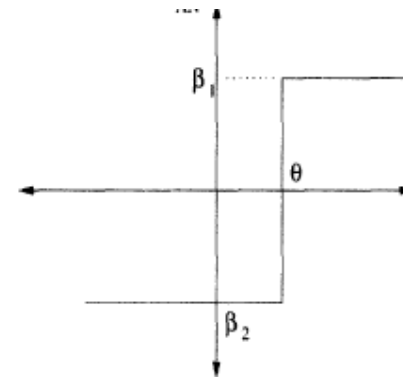
$$h(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}; h'(x) = [1 - h(x)][1 + h(x)]$$

# Step Function

$$y = f(\text{act}) = \begin{cases} \beta_1 & \text{If } \text{act} \geq 0 \\ \beta_2 & \text{If } \text{act} < 0 \end{cases}$$

For the step function only one of the two scalar values are possible at the output

Usually  $(\beta_1, \beta_2)$  are taken as  $(1, -1)$  or  $(1, 0)$

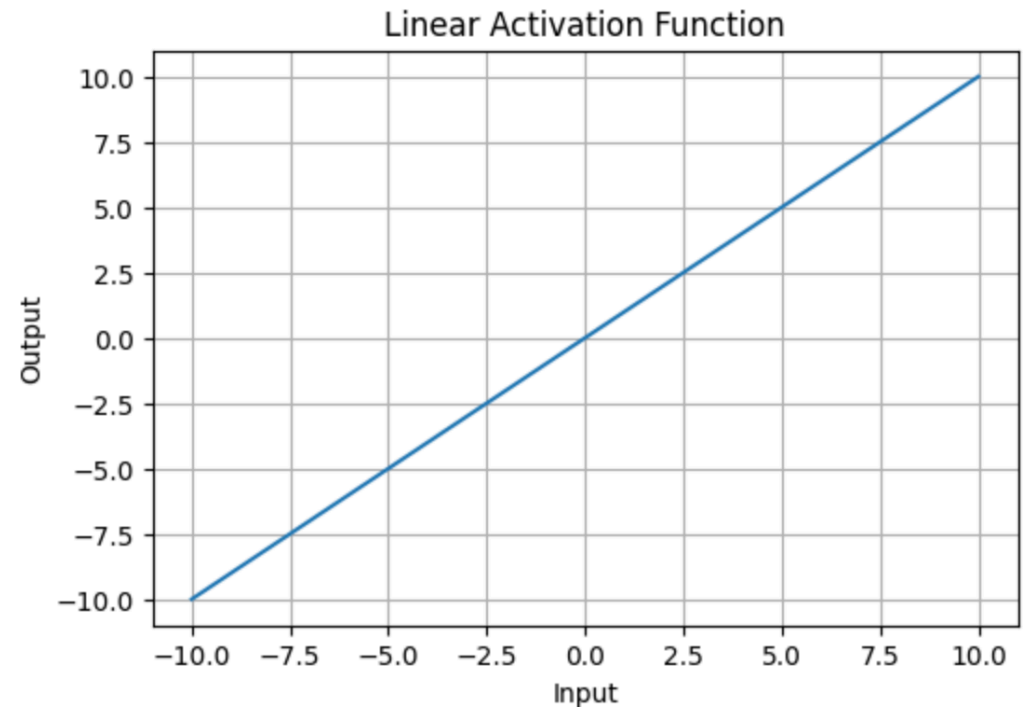


# Linear Function

$$y = f(\text{act}) = \gamma * \text{act}$$

The neuron output is simply equal to the weighted sum of the inputs. It may be modulated by a constant factor  $\gamma$

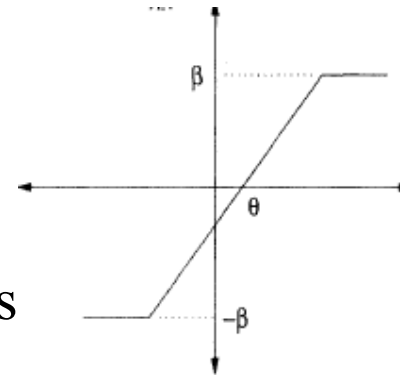
- Decision Boundary: This is not suitable for neurons used for classification purposes



# Ramp Function

$$\begin{aligned} y = f(\text{act}) &= \beta && \text{If } \text{act} \geq \beta \\ &= \text{act} && \text{If } -\beta < \text{act} < \beta \\ &= -\beta && \text{If } \text{act} \leq -\beta \end{aligned}$$

It is a combination of the linear and step functions





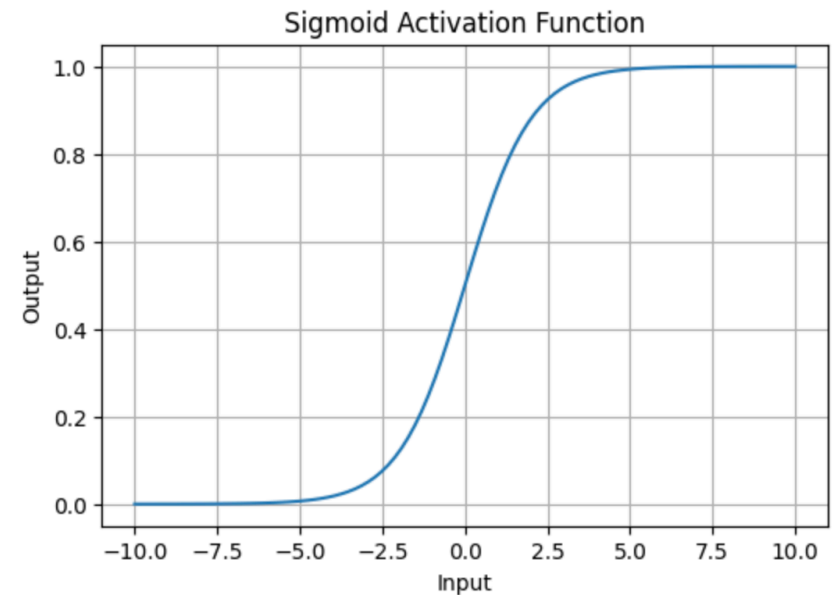
# Sigmoid Function (Logistic function)

$$y = f(\text{act}) = \frac{1}{1 + e^{-\lambda(\text{act})}}$$

The sigmoid function is a continuous version of the ramp function

The parameter  $\lambda$  controls the steepness of the function. Large  $\lambda$  makes it almost a unit step function.

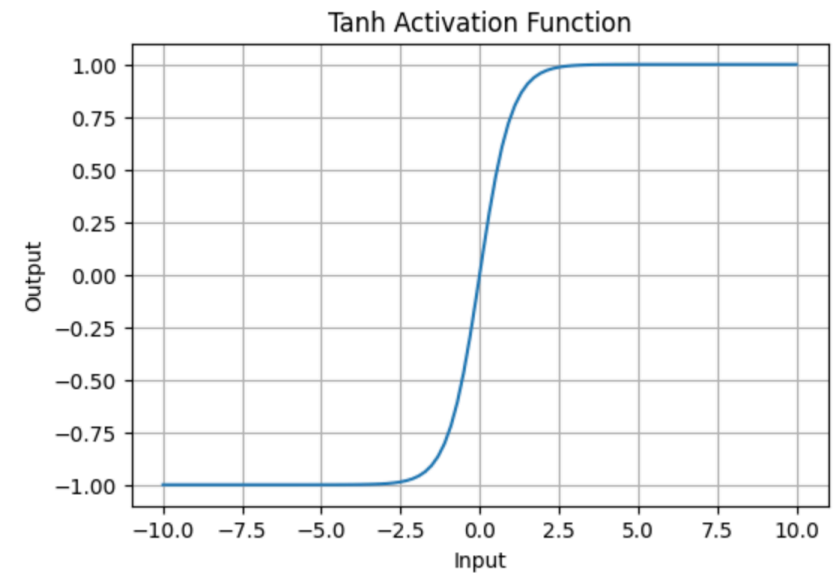
Usually  $\lambda = 1$



# Hyperbolic Tangent Function

$$y = f(\text{act}) = \frac{e^{\lambda(\text{act})} - e^{-\lambda(\text{act})}}{e^{\lambda(\text{act})} + e^{-\lambda(\text{act})}}$$
$$= \left[ \frac{2}{1 + e^{-\lambda(\text{act})}} \right] - 1$$

The output of this function is in the range  
(-1, 1)

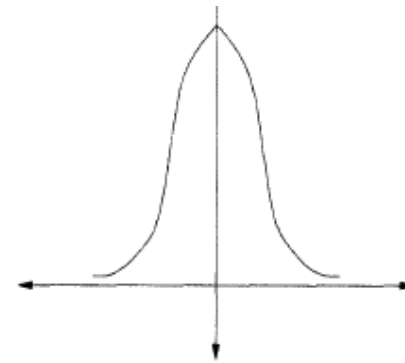


# Gaussian Function

$$y = f(\text{act}) = e^{-\theta}$$

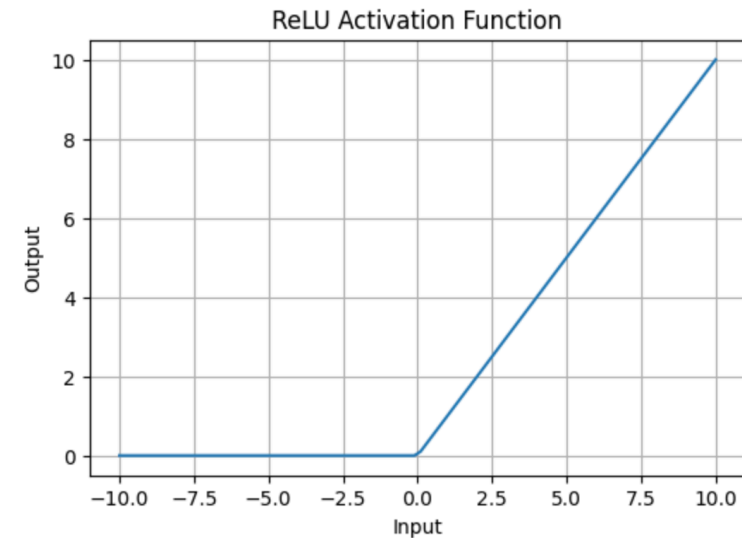
where  $\theta = (\text{act})^2/\sigma^2$

Where  $\sigma^2$  is the variance of the Gaussian distribution



# ReLU Function

- ReLU activation is defined by  $F(x) = \max(0, x)$
- This means that if the input  $x$  is positive, ReLU returns  $x$ , if the input is negative, it returns 0.



# Multi Layer Perceptron: Activation functions

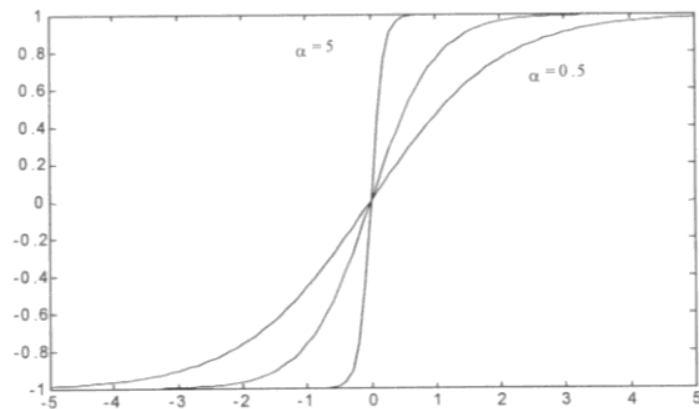
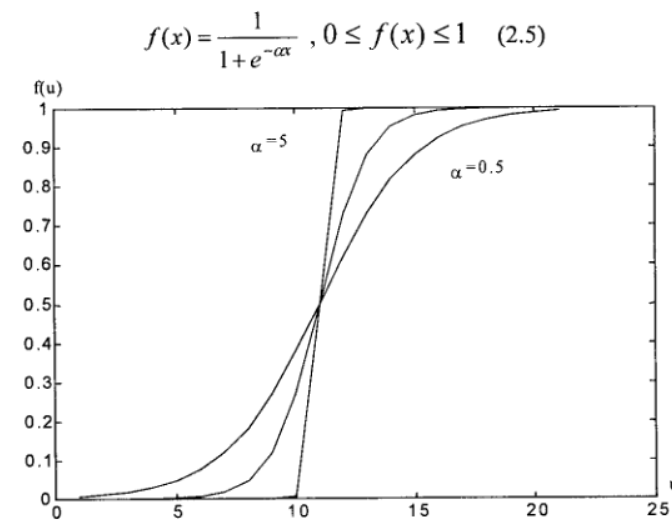
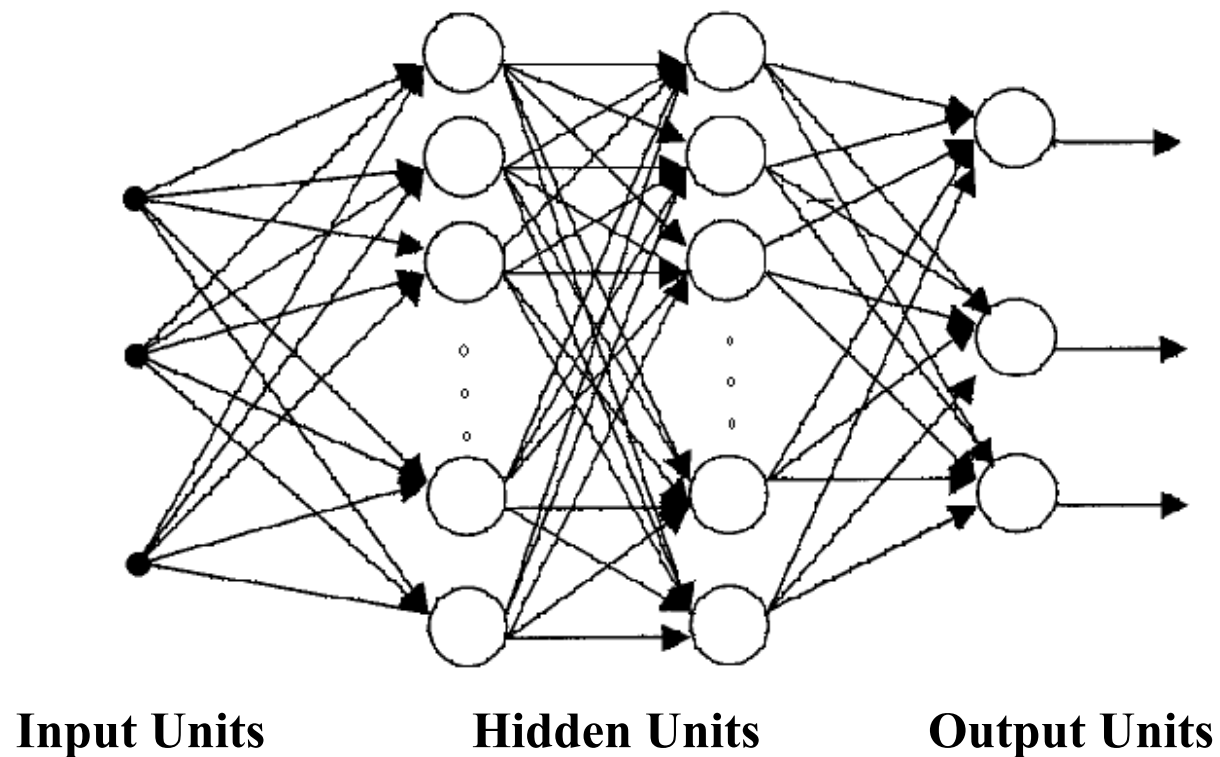


Figure 2.8: A Tangent Hyperbolic Activation Function.



# Multi Layer Perceptron: Architecture & Forward Pass

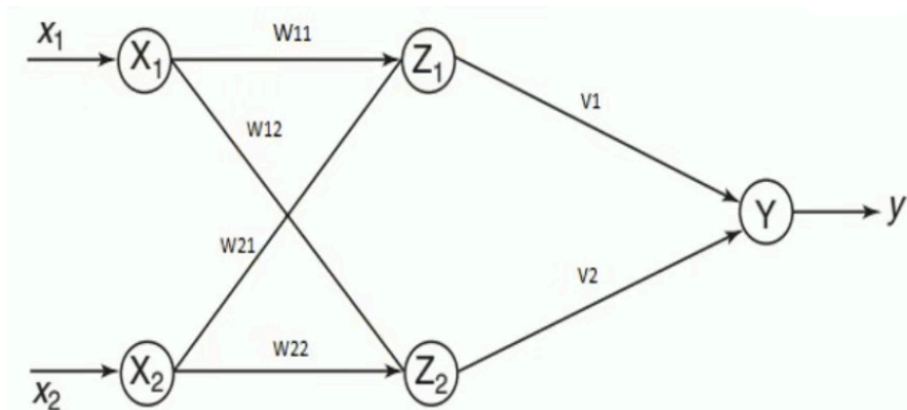


# Activity

Reference

[https://pse-ai-at-the-edge.h-da.io/ai-at-the-edge/AI-Models/Multi-Layer\\_Perceptron/Multi-Layer\\_Perceptron.html](https://pse-ai-at-the-edge.h-da.io/ai-at-the-edge/AI-Models/Multi-Layer_Perceptron/Multi-Layer_Perceptron.html)

- Design an XOR gate using MLP
- Show each iteration for weight and bias updates
- Explain the choice of activation function



| $x_1$ | $x_2$ | $y$ |
|-------|-------|-----|
| 0     | 0     | 0   |
| 0     | 1     | 1   |
| 1     | 0     | 1   |
| 1     | 1     | 0   |